Plataforma Web Colaborativa de Recetas de Cocina

Marcos Aguilar Marín

Andrey Flores Chavarria

Leidy Salazar Sánchez

Sebastián Sánchez Hernández

Licenciatura en Ingeniería Informática con énfasis en Desarrollo de Software

Universidad Latinoamericana de Ciencia y Tecnología (ULACIT)

Programación de Aplicaciones y Servicios Web (Proyecto Final)

Ing. Mauricio Rivera Villalobos

6 de octubre de 2026

Tabla de Contenido

Introducción 3
Planteamiento del Problema 4
Solución Propuesta 5
Marco Teórico 7
    Aplicaciones Web 7
    Arquitectura Cliente-Servidor 8
    Servicios Web y API REST 8
    Frontend con React 9
    Backend con C# y ASP.NET Core 9
    Base de Datos Relacional 10
    Autenticación y Autorización 11
Misión 11
Visión 11
Objetivo General 12
Objetivos Específicos 12
Requerimientos Funcionales 13
    Accesos por Actor 18
        Visitante 18
        Usuario 18
        Administrador 18
Requerimientos No Funcionales 19
Diagrama de Casos de Uso 22
    Herencia entre Actores 22
Referencias 24

Introducción

Las aplicaciones web han cambiado la forma en que realizamos muchas actividades cotidianas, pues reúnen en un solo lugar información y servicios que antes estaban dispersos. La cocina es un buen ejemplo: las recetas viven repartidas entre sitios web, redes sociales, videos y otras plataformas digitales, lo que complica organizarlas, consultarlas y guardarlas. Además, muchas de estas alternativas se limitan a mostrar contenido y no ofrecen un espacio donde las personas puedan publicar sus propias recetas e interactuar con las de otros.

Para responder a esta necesidad, el presente proyecto propone el desarrollo de una aplicación web de recetas de cocina que permita consultar, publicar, organizar y comentar recetas dentro de una misma plataforma. Cualquier visitante podrá explorar y buscar recetas; las personas registradas, además, podrán crear y administrar sus propias recetas, guardarlas como favoritas, indicar que les gustan, comentarlas y reportar contenido cuando sea necesario. Un rol administrativo se encargará de moderar el contenido y de gestionar los usuarios, las categorías y los reportes.

La aplicación se construirá con una arquitectura que separa la interfaz de usuario (*frontend*) de la lógica y la gestión de datos (*backend*). Para la interfaz utilizaremos React; para los servicios, C# con ASP.NET Core Web API; y para el almacenamiento, una base de datos relacional PostgreSQL, a la que accederemos mediante Entity Framework Core.

Con este proyecto buscamos entregar una solución web funcional y accesible que centralice las actividades relacionadas con las recetas de cocina y ofrezca una experiencia diferenciada según el tipo de usuario. Además, nos permite aplicar lo aprendido en el curso de Programación de Aplicaciones y Servicios Web: desarrollo de *frontend* y *backend*, servicios web, autenticación y autorización, persistencia de datos, seguridad y despliegue de aplicaciones.

Planteamiento del Problema

Hoy en día, Internet es una de las primeras fuentes a las que recurrimos para buscar recetas e información sobre la preparación de alimentos. Ese contenido está repartido entre sitios especializados, blogs, redes sociales, plataformas de video y otras fuentes digitales. Esta variedad facilita el acceso a una gran cantidad de recetas, pero también dispersa la información entre plataformas y dificulta organizarla y volver a consultarla.

Es común encontrar una receta en una red social, otra en un video y otras más en distintos sitios web, sin contar con un espacio central donde guardarlas, organizarlas y consultarlas. Lo mismo ocurre con quien desea compartir sus propias preparaciones: muchas veces depende de plataformas cuyo propósito principal no es crear y organizar recetas de cocina de forma estructurada.

Otra dificultad aparece al buscar una receta que responda a una necesidad concreta. Con tanto contenido disponible, resulta complicado localizar preparaciones según criterios como los ingredientes con que se cuenta, el tipo de comida, el tiempo de preparación o el nivel de dificultad. Un caso muy frecuente es tener ciertos ingredientes en casa y no saber qué se puede cocinar con ellos, lo que obliga a realizar múltiples búsquedas hasta encontrar una alternativa adecuada.

La interacción alrededor de las recetas también se encuentra fragmentada entre diferentes plataformas. Los comentarios, las recomendaciones, el contenido guardado y las valoraciones de otras personas ayudan a decidir qué preparar, pero rara vez están reunidos y organizados en un solo lugar.

Por todo lo anterior, surge la necesidad de contar con una plataforma web especializada que centralice la consulta, la organización y la publicación de recetas de cocina, que integre a

una comunidad de usuarios y que facilite el descubrimiento de preparaciones según distintos criterios y necesidades.

Solución Propuesta

Como respuesta a la problemática descrita, proponemos desarrollar una Plataforma Web Colaborativa de Recetas de Cocina que centralice la consulta, la publicación, la organización y la interacción con recetas. La plataforma busca ser un espacio especializado donde las personas descubran diferentes preparaciones y, al registrarse, participen activamente creando y administrando sus propias recetas.

El acceso dependerá del tipo de usuario. Los visitantes podrán navegar por la plataforma, consultar recetas y utilizar las herramientas de búsqueda y filtrado. Los usuarios registrados podrán, además, publicar sus propias recetas, modificarlas o eliminarlas, e interactuar con las publicaciones de otras personas mediante comentarios, “Me gusta” y la opción de guardar recetas como favoritas para consultarlas después.

Cada receta almacenará información estructurada que facilite su comprensión y su localización: nombre, descripción, ingredientes y cantidades, pasos de preparación, tiempo estimado, número de porciones, nivel de dificultad, categoría e imágenes. Esta estructura permitirá presentar la información de manera uniforme y utilizar distintos criterios para buscar y filtrar las recetas.

La aplicación también incluirá funciones de moderación y administración. Los administradores podrán gestionar categorías, supervisar usuarios y moderar las recetas o los comentarios que incumplan las normas de la plataforma. Los usuarios registrados, por su parte, podrán reportar contenido para que los administradores revisen los reportes y tomen las acciones

correspondientes. Como la moderación ocurre después de la publicación, no se requiere aprobar previamente cada receta.

Desde el punto de vista tecnológico, la plataforma se desarrollará con una arquitectura web que separa el *frontend*, el *backend* y la persistencia de datos. La interfaz de usuario se construirá con React; la lógica de negocio y los servicios web, con C# y ASP.NET Core Web API sobre .NET 10, la versión de soporte a largo plazo (LTS, por su sigla en inglés) vigente, con soporte hasta noviembre de 2028 (Microsoft, 2026c). La comunicación entre ambos componentes se realizará mediante una API REST, es decir, una interfaz de programación de aplicaciones (API) que sigue el estilo arquitectónico REST. Los datos se almacenarán en una base de datos relacional PostgreSQL, a la que se accederá con Entity Framework Core para gestionar la persistencia y las relaciones entre los datos. La Tabla 1 resume las tecnologías seleccionadas para el prototipo.

Tabla 1

Tecnologías Seleccionadas para el Prototipo

Capa	Tecnología	Función en el prototipo
Interfaz web (*frontend*)	React 19, con Vite como herramienta de construcción y React Router para la navegación	Vistas del catálogo, formularios de registro e inicio de sesión, creación y edición de recetas, perfiles, favoritos y administración.
Servicios (*backend*)	C# y ASP.NET Core 10 (Web API) sobre .NET 10 (LTS)	Lógica de negocio, validación de datos, control de acceso por rol y exposición de los *endpoints* de la API.
Comunicación	API REST sobre HTTP, con datos en formato JSON (*JavaScript Object Notation*)	Intercambio de información entre el *frontend* y el *backend*. El *backend* habilita CORS (intercambio de recursos de origen cruzado) para aceptar las solicitudes del *frontend*.
Base de datos	PostgreSQL 18	Almacenamiento de usuarios, recetas, ingredientes, categorías, comentarios, favoritos y reportes.
Acceso a datos	Entity Framework Core 10 con el proveedor Npgsql	Representación de las entidades como clases de C#, consultas y migraciones de la base de datos.

Capa	Tecnología	Función en el prototipo
Autenticación y autorización	JWT (*JSON Web Token*) y autorización por roles de ASP.NET Core	Verificación de la identidad, restricción de acciones según el rol (Visitante, Usuario o Administrador) y almacenamiento de contraseñas con *hash*.
Imágenes de recetas	Archivos almacenados en el servidor	La base de datos conserva únicamente la ubicación de cada imagen.

Nota. Las versiones corresponden a las versiones estables vigentes en octubre de 2026 (Meta Platforms, Inc., s. f.; Microsoft, 2026c; PostgreSQL Global Development Group, s. f.) y deberán confirmarse al iniciar el desarrollo del prototipo. El entorno de despliegue se definirá en una etapa posterior del proyecto.

Marco Teórico

El desarrollo de una plataforma web para publicar y consultar recetas exige integrar conceptos y tecnologías de distintas áreas: aplicaciones web, servicios web, gestión de información e interacción entre usuarios. En conjunto, permiten definir una arquitectura en la que la interfaz, la lógica de negocio y el almacenamiento de datos funcionan de manera coordinada.

Aplicaciones Web

Una aplicación web es un sistema de software al que se accede mediante un navegador y que permite realizar diversas operaciones a través de una interfaz gráfica. A diferencia de un sitio web orientado a presentar información, una aplicación web permite al usuario interactuar con el sistema: ingresar información, consultar datos y ejecutar acciones.

En este proyecto, la aplicación permitirá consultar y buscar recetas de cocina. Quienes se registren podrán, además, publicar y administrar sus propias recetas, comentar publicaciones, indicar que una receta les gusta y guardarlas como favoritas.

Arquitectura Cliente-Servidor

La arquitectura cliente-servidor distribuye las responsabilidades de una aplicación entre componentes diferenciados. El cliente presenta la interfaz y gestiona la interacción con el usuario; el servidor procesa las solicitudes, aplica las reglas de negocio y realiza las operaciones sobre los datos. Esta separación es una de las restricciones del estilo arquitectónico REST que describe Fielding (2000).

En la plataforma propuesta, el *frontend* actuará como cliente y se comunicará con el *backend* mediante servicios web. Como ambos componentes se ejecutarán como aplicaciones independientes, el *backend* deberá habilitar CORS para aceptar las solicitudes del *frontend* (Microsoft, 2026b). Esta separación mantiene ordenada la aplicación y permite desarrollar cada componente según sus responsabilidades.

Servicios Web y API REST

Los servicios web permiten que distintos componentes de una aplicación se comuniquen e intercambien información a través de una red. En este proyecto se desarrollará una API REST que actuará como intermediaria entre la interfaz de usuario y la lógica del sistema. REST es un estilo arquitectónico para sistemas distribuidos en la web propuesto por Fielding (2000).

La API permitirá realizar operaciones sobre usuarios, recetas, ingredientes, categorías, comentarios, favoritos, “Me gusta” y reportes, entre otras funcionalidades. Para ello se emplearán los métodos del protocolo HTTP (*Hypertext Transfer Protocol*): GET para consultar información, POST para registrar nuevos recursos, PUT para reemplazarlos y DELETE para eliminarlos (Fielding et al., 2022), además de PATCH para realizar modificaciones parciales.

La información entre el *frontend* y el *backend* se intercambiará en formato JSON, un formato de texto ligero e independiente del lenguaje de programación para el intercambio de datos (Bray, 2017), de uso muy extendido en las aplicaciones web.

Frontend con React

El *frontend* es el componente con el que interactúa directamente el usuario. Para construirlo utilizaremos React, una biblioteca de JavaScript para crear interfaces de usuario mediante componentes reutilizables (Meta Platforms, Inc., s. f.), en su versión 19, vigente a octubre de 2026.

Como crearemos una aplicación de una sola página sin un *framework* de servidor, el proyecto se iniciará con Vite como herramienta de construcción, una de las opciones que la documentación de React menciona para empezar un proyecto desde cero, y con React Router para la navegación entre vistas, la biblioteca de enrutamiento más popular para React (Meta Platforms, Inc., s. f.).

Con React será posible desarrollar las diferentes vistas de la plataforma: el catálogo de recetas, los formularios de registro e inicio de sesión, la creación y edición de recetas, los perfiles de usuario, los favoritos y las interfaces de las funciones administrativas.

Backend con C# y ASP.NET Core

El *backend* procesa las solicitudes que llegan del *frontend*, implementa la lógica de negocio, controla el acceso a las funcionalidades y gestiona la comunicación con la base de datos.

Para este componente se utilizará C# con ASP.NET Core Web API, que permite desarrollar los *endpoints* mediante los cuales el *frontend* interactúa con los recursos de la aplicación. ASP.NET Core ofrece dos enfoques para construir APIs HTTP: Minimal APIs y APIs

basadas en controladores (Microsoft, 2026a); el equipo definirá cuál emplear antes de iniciar el prototipo. El desarrollo se hará sobre .NET 10, versión con soporte a largo plazo hasta noviembre de 2028 (Microsoft, 2026c).

El *backend* también aplicará las restricciones propias de cada tipo de usuario: Visitante, Usuario y Administrador.

Base de Datos Relacional

Las bases de datos relacionales almacenan información estructurada en tablas relacionadas entre sí. Este modelo es apropiado para la plataforma por las relaciones que existen entre elementos como usuarios, recetas, ingredientes, categorías, comentarios, favoritos y reportes.

Utilizaremos PostgreSQL como sistema gestor de base de datos; su versión mayor estable vigente es la 18 (PostgreSQL Global Development Group, s. f.). Por ejemplo, un usuario podrá publicar múltiples recetas, una receta podrá contener múltiples ingredientes y diferentes usuarios podrán guardar distintas recetas como favoritas. Estas relaciones se representarán mediante claves primarias, claves foráneas y tablas intermedias cuando sea necesario.

Para comunicar el *backend* con PostgreSQL se utilizará Entity Framework Core, un mapeador objeto-relacional que permite trabajar con la base de datos mediante objetos de .NET (Microsoft, s. f.). Como Entity Framework Core accede a cada motor de base de datos mediante un proveedor, se empleará el proveedor Npgsql.EntityFrameworkCore.PostgreSQL. De esta forma, las entidades de la base de datos se representarán como clases de C# y las operaciones de persistencia se gestionarán desde la aplicación.

Autenticación y Autorización

La autenticación verifica la identidad de un usuario, mientras que la autorización determina las acciones que ese usuario puede realizar dentro del sistema (Microsoft, 2026b). Ambos conceptos son necesarios por los distintos niveles de acceso establecidos para la plataforma.

Para implementarlos proponemos que la API emita un JWT (*JSON Web Token*) cuando una persona inicie sesión. Un JWT es una forma compacta de representar información que se transfiere entre dos partes (Jones et al., 2015); el *frontend* lo enviará en cada solicitud protegida y el *backend* lo validará junto con el rol del usuario.

Los visitantes podrán consultar el contenido público sin autenticarse. Los usuarios registrados tendrán acceso a las funciones de publicación e interacción con las recetas, y los administradores contarán con permisos de moderación y administración de la plataforma. En consecuencia, ciertas operaciones deberán restringirse según la identidad, el rol y los permisos de quien realiza la solicitud.

Misión

La misión de la aplicación es ofrecer una plataforma web accesible y organizada para consultar, publicar y gestionar recetas de cocina, de modo que las personas compartan sus preparaciones, descubran nuevo contenido e interactúen con una comunidad interesada en la cocina.

Visión

La visión de la aplicación es consolidarse como una plataforma digital de recetas que ofrezca una experiencia práctica y colaborativa, con herramientas que faciliten el

descubrimiento, la organización y el intercambio de preparaciones culinarias, y que evolucione mediante nuevas funcionalidades según las necesidades de sus usuarios.

Objetivo General

Desarrollar una aplicación web de recetas de cocina que permita consultar, publicar, organizar e interactuar con contenido culinario en una plataforma centralizada, mediante una arquitectura basada en una interfaz web, servicios de *backend* y una base de datos relacional.

Objetivos Específicos

Los objetivos específicos del proyecto son los siguientes:

- Implementar una interfaz web interactiva para consultar, buscar, visualizar y gestionar recetas, de acuerdo con los niveles de acceso establecidos en la plataforma.
- Construir los servicios y la lógica de negocio necesarios para gestionar usuarios, recetas, ingredientes, categorías, interacciones y moderación de contenido, con mecanismos de autenticación y autorización.
- Estructurar una base de datos relacional que almacene y relacione la información de usuarios, recetas, ingredientes, categorías, interacciones y reportes de la plataforma.

Requerimientos Funcionales

Los requerimientos funcionales describen las acciones, los procesos y los servicios que la aplicación deberá ofrecer a sus actores. La plataforma contempla tres actores principales: Visitante, Usuario (persona con una cuenta registrada) y Administrador, cada uno con distintos niveles de acceso y permisos. La Tabla 2 presenta los 21 requerimientos funcionales.

Tabla 2

Matriz de Requerimientos Funcionales

ID	Requerimiento	Descripción	Actor	Prioridad	Entrada	Proceso	Salida	Criterio de aceptación	Observaciones
RF-01	Registrar usuario	Permitir la creación de una nueva cuenta dentro de la plataforma.	Visitante	Alta	Nombre, correo electrónico, contraseña y confirmación de contraseña.	Validar los datos ingresados, verificar la disponibilidad del correo y registrar la cuenta.	Cuenta creada o mensaje de error.	La cuenta se crea cuando los datos son válidos y el correo no se encuentra registrado.	La contraseña deberá almacenarse de forma segura.
RF-02	Iniciar sesión	Permitir la autenticación de usuarios registrados y administradores.	Usuario / Administrador	Alta	Correo electrónico y contraseña.	Validar las credenciales y determinar el rol asociado a la cuenta.	Sesión iniciada o mensaje de credenciales inválidas.	El acceso se concede únicamente con credenciales válidas y una cuenta habilitada.	Los permisos dependerán del rol del usuario.
RF-03	Cerrar sesión	Permitir finalizar la sesión activa de un usuario autenticado.	Usuario / Administrador	Alta	Solicitud de cierre de sesión.	Finalizar la sesión activa e invalidar el acceso autenticado.	Sesión finalizada.	Después del cierre, las funcionalidades protegidas requieren una nueva autenticación.	—

ID	Requerimiento	Descripción	Actor	Prioridad	Entrada	Proceso	Salida	Criterio de aceptación	Observaciones
RF-04	Gestionar perfil	Permitir consultar y modificar la información del perfil del usuario.	Usuario / Administrador	Media	Nombre visible, fotografía, descripción y demás información modificable.	Mostrar los datos actuales, validar los cambios y actualizar la información.	Perfil actualizado o mensaje de error.	Los cambios válidos se almacenan y se muestran correctamente.	Los datos de autenticación podrán manejarse mediante procesos independientes.
RF-05	Consultar recetas propias	Mostrar las recetas publicadas por el usuario autenticado.	Usuario / Administrador	Alta	Solicitud de consulta.	Identificar al usuario y recuperar las recetas asociadas a su cuenta.	Listado de recetas propias.	Se muestran correctamente las recetas publicadas por el usuario autenticado.	Desde este apartado se podrá acceder a la edición y a la eliminación.
RF-06	Consultar recetas	Permitir visualizar las recetas disponibles en la plataforma y acceder a su información detallada.	Visitante / Usuario / Administrador	Alta	Solicitud de consulta o selección de una receta.	Recuperar la información de las recetas o de la receta seleccionada.	Listado o detalle de la receta.	La información de las recetas disponibles se muestra correctamente.	Es una funcionalidad pública.
RF-07	Buscar y filtrar recetas	Permitir localizar recetas mediante criterios de búsqueda y filtrado.	Visitante / Usuario / Administrador	Alta	Texto de búsqueda, filtros seleccionados o ambos.	Comparar los criterios ingresados con la información de las recetas disponibles.	Listado de recetas coincidentes.	Los resultados corresponden con los criterios seleccionados por el usuario.	Se podrán utilizar criterios como categoría, dificultad o tiempo de preparación.

ID	Requerimiento	Descripción	Actor	Prioridad	Entrada	Proceso	Salida	Criterio de aceptación	Observaciones
RF-08	Crear receta	Permitir que un usuario publique una nueva receta.	Usuario / Administrador	Alta	Nombre, descripción, ingredientes, cantidades, pasos, categoría, dificultad, tiempo, porciones e imagen.	Validar la información, asociar la receta con el usuario y almacenarla.	Receta publicada o mensaje de error.	Una receta con todos los campos obligatorios válidos queda disponible en la plataforma.	No requiere aprobación previa del administrador.
RF-09	Editar receta propia	Permitir modificar la información de una receta publicada por el propio usuario.	Usuario / Administrador	Alta	Receta seleccionada y nuevos datos.	Verificar propiedad, validar los cambios y actualizar la receta.	Receta actualizada o mensaje de error.	Los cambios válidos quedan reflejados en la receta correspondiente.	Un usuario no puede editar recetas pertenecientes a otros usuarios.
RF-10	Eliminar receta propia	Permitir eliminar una receta publicada por el propio usuario.	Usuario / Administrador	Alta	Receta seleccionada y confirmación.	Verificar propiedad y realizar la eliminación correspondiente.	Confirmación de eliminación.	La receta deja de estar disponible después de confirmar la operación.	Debe solicitarse confirmación antes de eliminar.
RF-11	Marcar receta como favorita	Permitir almacenar una receta dentro de los favoritos del usuario.	Usuario / Administrador	Media	Receta seleccionada.	Asociar la receta con la lista de favoritos del usuario.	Receta agregada a favoritos.	La receta seleccionada aparece en los favoritos del usuario.	Una misma receta no deberá duplicarse en favoritos.
RF-12	Gestionar recetas favoritas	Permitir consultar y eliminar recetas almacenadas como favoritas.	Usuario / Administrador	Media	Solicitud de consulta o selección de una receta favorita.	Recuperar los favoritos o eliminar la asociación seleccionada.	Listado actualizado de favoritos.	El usuario puede consultar sus favoritos y retirar cualquiera de ellos.	La eliminación de favoritos no elimina la receta original.

ID	Requerimiento	Descripción	Actor	Prioridad	Entrada	Proceso	Salida	Criterio de aceptación	Observaciones
RF-13	Dar “Me gusta” a una receta	Permitir expresar aprobación sobre una receta publicada.	Usuario / Administrador	Media	Receta seleccionada.	Registrar o retirar el “Me gusta” asociado al usuario y a la receta.	Estado y contador de “Me gusta” actualizados.	Cada usuario puede mantener como máximo un “Me gusta” activo por receta.	El usuario podrá retirar posteriormente su “Me gusta”.
RF-14	Comentar receta	Permitir publicar comentarios en las recetas disponibles.	Usuario / Administrador	Media	Receta seleccionada y contenido del comentario.	Validar el contenido y asociar el comentario con la receta y el usuario.	Comentario publicado o mensaje de error.	Un comentario válido se muestra asociado a la receta y al usuario que lo publicó.	El comentario deberá respetar las restricciones establecidas por la plataforma.
RF-15	Gestionar comentario propio	Permitir modificar o eliminar comentarios realizados por el propio usuario.	Usuario / Administrador	Media	Comentario seleccionado y acción solicitada.	Verificar la autoría y actualizar o eliminar el comentario.	Comentario actualizado o eliminado.	Solamente el autor puede modificar o eliminar su propio comentario, salvo las acciones de moderación.	El administrador podrá eliminar comentarios mediante su función de moderación.
RF-16	Reportar contenido	Permitir reportar recetas o comentarios considerados inapropiados.	Usuario	Media	Contenido seleccionado, motivo y descripción opcional.	Registrar el reporte y asociarlo con el contenido y usuario correspondiente.	Confirmación del reporte.	El reporte queda disponible para revisión administrativa.	El reporte no elimina automáticamente el contenido.
RF-17	Gestionar categorías	Permitir administrar las categorías utilizadas para clasificar recetas.	Administrador	Alta	Datos de categoría y acción solicitada.	Crear, modificar o eliminar categorías según corresponda.	Catálogo de categorías actualizado.	Los cambios realizados se reflejan en las opciones de clasificación disponibles.	Deben considerarse las recetas asociadas antes de eliminar una categoría.

ID	Requerimiento	Descripción	Actor	Prioridad	Entrada	Proceso	Salida	Criterio de aceptación	Observaciones
RF-18	Moderar recetas	Permitir revisar y retirar recetas que incumplan las reglas de la plataforma.	Administrador	Alta	Receta seleccionada y acción de moderación.	Revisar la receta y ejecutar la acción administrativa correspondiente.	Receta mantenida o retirada de la plataforma.	Una receta retirada mediante moderación deja de estar disponible públicamente.	La moderación ocurre posteriormente a la publicación.
RF-19	Moderar comentarios	Permitir retirar comentarios que incumplan las reglas de la plataforma.	Administrador	Alta	Comentario seleccionado y acción de moderación.	Revisar el comentario y ejecutar la acción correspondiente.	Comentario mantenido o retirado.	Un comentario retirado mediante moderación deja de visualizarse públicamente.	Puede originarse a partir de un reporte.
RF-20	Gestionar reportes	Permitir consultar y atender los reportes generados por los usuarios.	Administrador	Alta	Reporte seleccionado y decisión administrativa.	Consultar el contenido reportado, analizar el motivo y registrar la acción correspondiente.	Reporte atendido y estado actualizado.	El administrador puede consultar reportes pendientes y registrar su resolución.	La resolución puede implicar mantener o moderar el contenido reportado.
RF-21	Gestionar usuarios	Permitir administrar el estado de las cuentas registradas en la plataforma.	Administrador	Alta	Usuario seleccionado y acción administrativa.	Consultar la cuenta y suspenderla o reactivarla según corresponda.	Estado de la cuenta actualizado.	Una cuenta suspendida no puede acceder a las funcionalidades que requieren autenticación.	El administrador no modificará arbitrariamente los datos personales del usuario.

Nota. El guion largo (—) indica que el requerimiento no tiene observaciones.

Accesos por Actor

Visitante

El Visitante accede únicamente a las funcionalidades públicas: puede consultar el catálogo de recetas, ver el detalle de cada una, utilizar el buscador y aplicar los filtros disponibles. También puede iniciar el proceso de registro para convertirse en usuario de la plataforma.

Usuario

El Usuario cuenta con las capacidades del Visitante y, además, puede administrar su perfil y sus propias recetas, interactuar con las recetas de otras personas mediante favoritos, “Me gusta” y comentarios, y reportar contenido que considere inapropiado.

Administrador

El Administrador accede a las funcionalidades del Usuario y dispone, además, de herramientas administrativas para gestionar usuarios y categorías, moderar recetas y comentarios, y atender los reportes generados por los usuarios.

Requerimientos No Funcionales

Los requerimientos no funcionales describen las características de calidad y las condiciones técnicas bajo las cuales deberá operar la aplicación. Complementan las funcionalidades definidas anteriormente y establecen criterios como seguridad, rendimiento, disponibilidad, usabilidad, compatibilidad y mantenibilidad. La Tabla 3 presenta los 12 requerimientos no funcionales.

Tabla 3

Matriz de Requerimientos No Funcionales

ID	Categoría	Descripción detallada	Métrica / Indicador	Nivel esperado	Prioridad	Restricción asociada	Método de validación	Observaciones
RNF-01	Seguridad	La aplicación deberá proteger las contraseñas de los usuarios mediante mecanismos seguros de almacenamiento.	Contraseñas almacenadas como texto plano.	0 contraseñas almacenadas en texto plano.	Alta	Las contraseñas no podrán almacenarse directamente en la base de datos sin protección.	Inspección de la base de datos y pruebas de seguridad.	Se deberá utilizar un mecanismo de hash apropiado para contraseñas.
RNF-02	Seguridad	La aplicación deberá restringir las funcionalidades protegidas de acuerdo con el estado de autenticación y el rol del usuario.	Accesos no autorizados permitidos.	0 accesos no autorizados en las pruebas definidas.	Alta	Las operaciones protegidas deberán validarse desde el backend.	Pruebas de autorización con los diferentes roles.	Se contemplan Visitante, Usuario y Administrador.
RNF-03	Seguridad	La aplicación deberá validar la información recibida antes de procesarla o almacenarla.	Entradas inválidas aceptadas durante las pruebas.	0 entradas inválidas aceptadas en los casos de prueba definidos.	Alta	La validación no deberá depender exclusivamente del frontend.	Pruebas con datos válidos, inválidos y campos obligatorios vacíos.	Las validaciones deberán realizarse también desde el backend.

ID	Categoría	Descripción detallada	Métrica / Indicador	Nivel esperado	Prioridad	Restricción asociada	Método de validación	Observaciones
RNF-04	Rendimiento	La aplicación deberá responder a las operaciones comunes dentro de un tiempo adecuado bajo condiciones normales de uso.	Tiempo de respuesta de las solicitudes principales.	Máximo 3 segundos en condiciones normales de prueba.	Media	Depende de la conexión de red y de la disponibilidad de los servicios utilizados.	Medición de tiempos durante pruebas funcionales.	Se consideran principalmente consultas, autenticación y operaciones CRUD.
RNF-05	Usabilidad	La interfaz deberá presentar una navegación comprensible y consistente entre las diferentes secciones.	Finalización correcta de tareas básicas durante pruebas de usuario.	Al menos 90 % de las tareas de prueba completadas sin asistencia.	Alta	Las funciones principales deberán ser identificables desde la interfaz.	Pruebas de usabilidad con tareas previamente definidas.	Se evaluarán acciones como buscar, consultar y publicar una receta.
RNF-06	Diseño adaptable	La interfaz deberá adaptarse a diferentes tamaños de pantalla para permitir su utilización desde computadoras, tabletas y dispositivos móviles.	Visualización correcta en resoluciones seleccionadas.	Correcta visualización y operación en las resoluciones de prueba definidas.	Media	El diseño deberá utilizar técnicas de interfaz adaptable.	Pruebas manuales mediante diferentes dimensiones de pantalla.	No implica el desarrollo de una aplicación móvil nativa.
RNF-07	Compatibilidad	La aplicación deberá funcionar correctamente en navegadores web modernos utilizados por los usuarios.	Navegadores compatibles durante las pruebas.	Funcionamiento correcto en las versiones actuales de Chrome, Edge y Firefox.	Media	El navegador deberá soportar las tecnologías web utilizadas por React.	Ejecución de casos de prueba en cada navegador definido.	Se probarán principalmente las funcionalidades esenciales.
RNF-08	Disponibilidad	La aplicación deberá permanecer accesible mientras los servicios necesarios para su funcionamiento se encuentren disponibles.	Acceso satisfactorio durante las pruebas de disponibilidad.	Acceso satisfactorio durante el periodo de pruebas planificado.	Media	Depende de la disponibilidad del servidor, del backend y de la base de datos.	Pruebas periódicas de acceso durante el periodo definido.	No se establece disponibilidad 24/7 debido al alcance académico del proyecto.

ID	Categoría	Descripción detallada	Métrica / Indicador	Nivel esperado	Prioridad	Restricción asociada	Método de validación	Observaciones
RNF-09	Integridad de datos	La aplicación deberá mantener la consistencia de las relaciones entre los datos almacenados.	Registros con referencias inválidas o inconsistentes.	0 inconsistencias en los casos de prueba definidos.	Alta	La base de datos utilizará relaciones y restricciones de integridad.	Pruebas CRUD e inspección de las relaciones almacenadas.	Especialmente importante para usuarios, recetas, ingredientes, comentarios y categorías.
RNF-10	Mantenibilidad	El código fuente deberá mantener una separación clara entre interfaz, lógica de negocio y acceso a datos.	Organización de los componentes y responsabilidades.	Componentes organizados según la arquitectura definida para el proyecto.	Media	El frontend y backend serán desarrollados como componentes separados.	Revisión de estructura y código fuente.	Facilita modificaciones y mantenimiento durante el desarrollo.
RNF-11	Manejo de errores	La aplicación deberá gestionar los errores de forma controlada y presentar mensajes comprensibles al usuario cuando corresponda.	Errores no controlados durante los casos de prueba.	0 errores no controlados en los casos de prueba definidos.	Media	No deberá mostrarse al usuario información técnica sensible del sistema.	Pruebas que provoquen situaciones de error conocidas.	Los detalles técnicos podrán registrarse internamente para diagnóstico.
RNF-12	Persistencia	La información registrada deberá conservarse correctamente después de finalizar una sesión o reiniciar los componentes de la aplicación.	Datos perdidos después de operaciones exitosas.	0 pérdidas de datos confirmados durante las pruebas.	Alta	Las operaciones exitosas deberán persistirse en PostgreSQL.	Registro, reinicio y posterior consulta de datos de prueba.	Aplica a usuarios, recetas, comentarios, favoritos y demás información persistente.

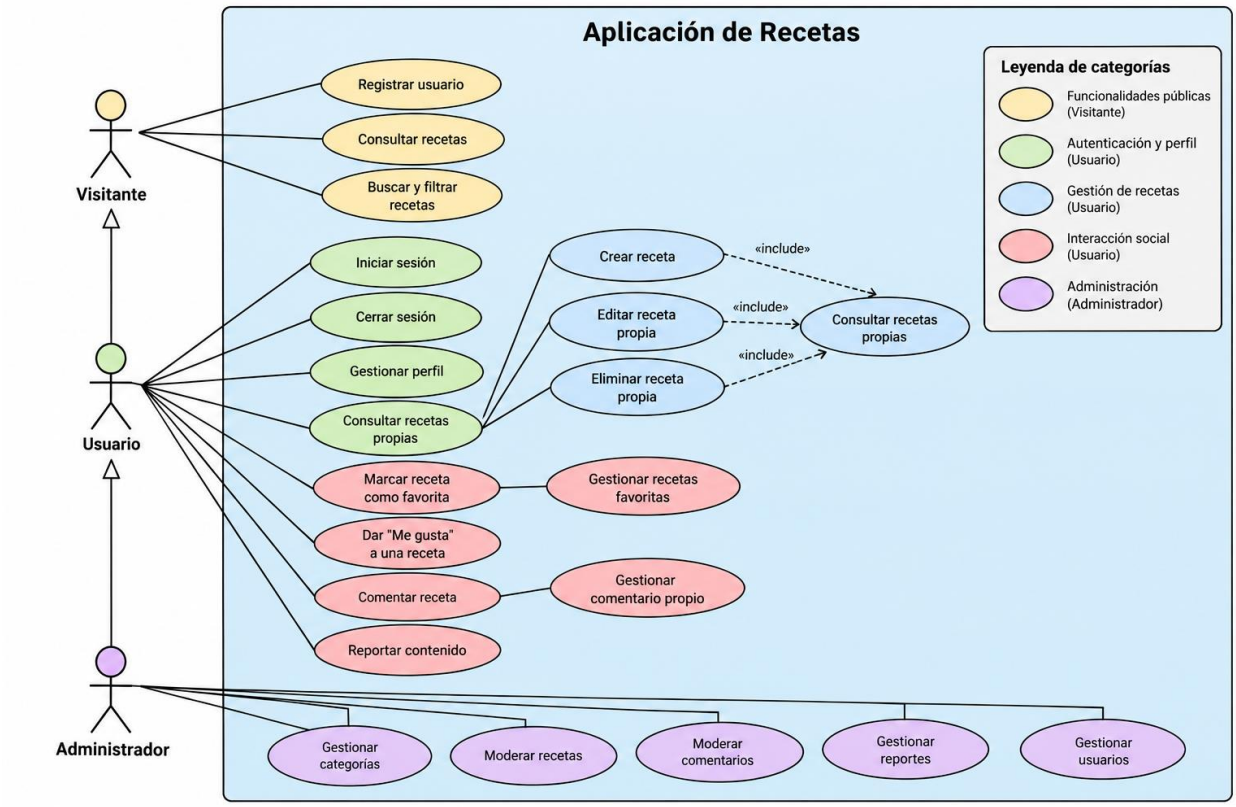
Nota. CRUD = crear, leer, actualizar y eliminar (*create, read, update, delete*).

Diagrama de Casos de Uso

El diagrama de casos de uso (Figura 1) representa las principales interacciones entre los actores identificados y las funcionalidades disponibles en la aplicación. Se contemplan tres actores: Visitante, Usuario y Administrador.

Figura 1

Diagrama de Casos de Uso de la Plataforma Colaborativa de Recetas



Nota. Elaboración propia.

Herencia entre Actores

El diagrama utiliza una relación de herencia, o generalización, entre los actores Visitante, Usuario y Administrador. Esta relación indica que un actor con mayor nivel de acceso conserva las funcionalidades del actor del que hereda y añade las suyas.

Así, el Usuario hereda las funcionalidades del Visitante (consultar, buscar y filtrar recetas) y suma las propias de una cuenta registrada. A su vez, el Administrador hereda las del Usuario y dispone de herramientas exclusivas de gestión y moderación: administración de usuarios, categorías, reportes, recetas y comentarios.

La generalización simplifica el diagrama, porque evita repetir las mismas asociaciones para cada actor, y muestra con claridad la jerarquía de permisos de la aplicación.

Referencias

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC 8259). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc8259

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fielding, R. T., Nottingham, M. y Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc9110

Jones, M. B., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc7519

Meta Platforms, Inc. (s. f.). *Creating a React app*. React. Recuperado el 3 de octubre de 2026, de https://react.dev/learn/creating-a-react-app

Microsoft. (s. f.). *Overview of Entity Framework Core*. Microsoft Learn. Recuperado el 3 de octubre de 2026, de https://learn.microsoft.com/ef/core/

Microsoft. (2026a). *APIs overview*. Microsoft Learn. Recuperado el 3 de octubre de 2026, de https://learn.microsoft.com/aspnet/core/fundamentals/apis

Microsoft. (2026b). *ASP.NET Core security topics*. Microsoft Learn. Recuperado el 3 de octubre de 2026, de https://learn.microsoft.com/aspnet/core/security/

Microsoft. (2026c). *.NET support policy*. .NET. Recuperado el 3 de octubre de 2026, de https://dotnet.microsoft.com/en-us/platform/support/policy

PostgreSQL Global Development Group. (s. f.). *Versioning policy*. PostgreSQL. Recuperado el 3 de octubre de 2026, de https://www.postgresql.org/support/versioning/